YZ 203 · VISUAL PROGRAMMING

Week 4

Graphics & paintComponent: drawing our own game

Erciyes University

Java Swing & Event-Driven Programming

Murat AKPULAT, PhD

What's on today

The label trick had a good run. Today we learn to draw, and the Timer gets a new job.

01
Why draw?
Limits of the colored-label trick

02
paintComponent
Inheritance, override, and who calls whom

03
Graphics API
Colors, shapes, lines, text

04
Timer + repaint()
Same Timer, a different job than Week 3

05
Egg draws itself
draw(Graphics) and a drawn basket

06
Falling Egg v1
v0 rebuilt with real graphics

Quick Recap: Where We Left Off

Week 2

Nine components, four listener interfaces.

Week 3

javax.swing.Timer fires an ActionEvent every N ms; a lambda moves a number and calls setLocation().

Falling Egg v0

A colored JLabel falls, a button pauses, a slider changes the delay.

The limit

A label is a rectangle. We cannot make an oval egg, a basket, or a smiley.

Today the egg stops being a component and becomes a drawing.

Why Do We Need to Draw?

Components are ready-made parts. A game needs shapes no component provides.

With components (Week 3)

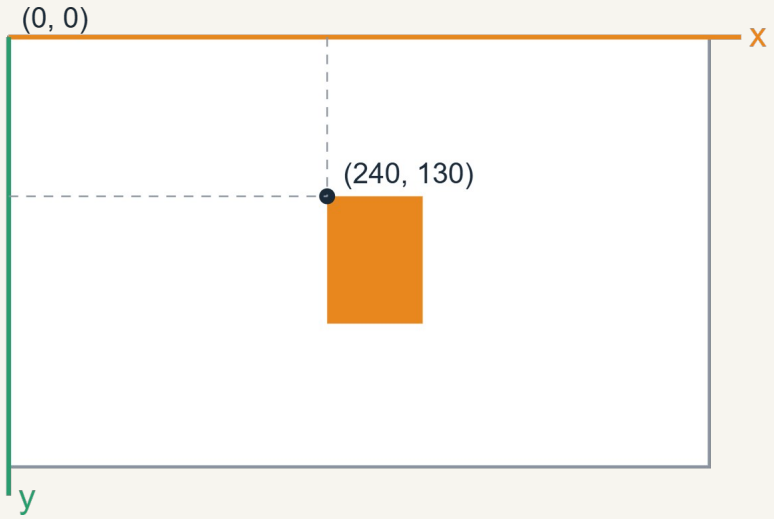
- Everything is a rectangle
- One component per object: 100 eggs = 100 JLabels
- Moving means asking each component to move
- No ovals, arcs, curves or lines

With Graphics (today)

- Any shape, any color, any position
- One panel paints every object
- Moving means changing numbers, then repaint
- Drawing code is plain Java methods

The Coordinate System

You met it in Week 3 with setLocation(). Drawing uses exactly the same grid.



(0, 0) is the top-left corner of the panel.

x grows to the right, **y** grows downward.

A shape is placed by its **top-left corner** plus a width and a height.

Your First Drawing

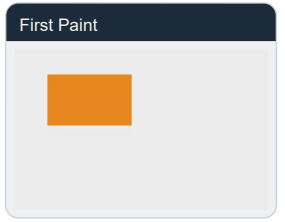
Scenario: an orange rectangle drawn by our own panel. No JLabel, no setBounds.

```java
import javax.swing.*;
import java.awt.*;

class GamePanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillRect(40, 30, 100, 60);
    }
}

public class FirstPaint {
    public static void main(String[] args) {
        JFrame f = new JFrame("First Paint");
        f.add(new GamePanel());
        f.setSize(300, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

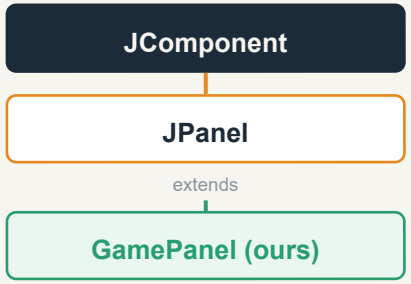
1. Inheritance — GamePanel extends JPanel: it is a panel, plus our own painting.

2. Override — we replace paintComponent. @Override makes the compiler catch typos in the name.

3. Callback — we never call paintComponent. Swing calls it whenever the panel must be drawn.

OOP 1: Inheritance, We Write Our Own Panel

In Week 1 we only used classes. Today we **extend** one.



Inherited for free: add(), setBackground(), getWidth(), getHeight(), repaint(), setPreferredSize()...

What we add: our own version of paintComponent(Graphics g).

Is-a test: a GamePanel *is a* JPanel, so frame.add(new GamePanel()) just works.

OOP 2: Overriding, Our Version Runs

JPanel already has a paintComponent. GamePanel replaces it with its own.

JPanel's version

```
// inside JPanel
void paintComponent(Graphics g) {
    // fill the background
}
```

Clears the panel with its background color.

→

GamePanel's version

```
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g); // 1. clear
    g.fillOval(x, y, 30, 40); // 2. draw
}
```

super.paintComponent(g) runs the parent's version first. Then we draw on top.

Polymorphism: Swing holds a JPanel reference, but at run time it calls the method of the real object, our GamePanel.

OOP 3: "Don't Call Us, We'll Call You"

The same idea as listeners, in a new place. You write it; Swing decides when it runs.

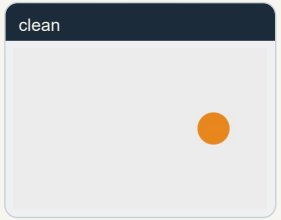
	actionPerformed (Weeks 1 to 3)	paintComponent (today)
What you write	a listener method	an overridden method
Who calls it	Swing, after an event	Swing, when the panel needs drawing
When	click, tick, selection	first shown, resized, uncovered, repaint()
You may call it yourself?	No, wait for the event	No. Call repaint() and let Swing do it

This is called a **callback**: code you hand to a framework so that it can call you back. Almost every GUI library works this way.

Why super.paintComponent(g) Matters

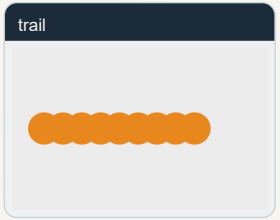
A moving shape drawn *without* clearing leaves a trail of old frames.

With super.paintComponent(g)



Each frame starts from a clean panel.

Without it



Old frames stay and pile up.

Rule: the first line of every paintComponent is super.paintComponent(g);

The Graphics Toolbox

The Graphics object g is the pen. You choose a color, then draw.

method	draws	the numbers mean
g.setColor(c)	nothing, picks the pen color	c is a Color
g.fillRect(x, y, w, h)	filled rectangle	top-left corner, width, height
g.drawRect(x, y, w, h)	rectangle outline	same
g.fillOval(x, y, w, h)	filled oval or circle	the box that contains it
g.drawLine(x1, y1, x2, y2)	a line	start point, end point
g.drawString(s, x, y)	text	x, and y of the text baseline
g.fillArc(x, y, w, h, start, sweep)	a pie slice or half oval	angles in degrees

Colors: The Pen Stays Until You Change It

```
g.setColor(Color.RED);
g.fillOval(20, 20, 40, 40);   // red
g.fillOval(80, 20, 40, 40);   // still red

g.setColor(new Color(255, 128, 0));
g.fillRect(20, 80, 100, 30);  // orange
```

setColor is state: every shape after it uses that color, until the next setColor.

- Color.RED
- Color.YELLOW
- Color.GREEN
- new Color(255, 128, 0)
- new Color(139, 90, 43)

new Color(r, g, b): three numbers from 0 to 255 for red, green and blue.

Shapes and Colors: a Traffic Light

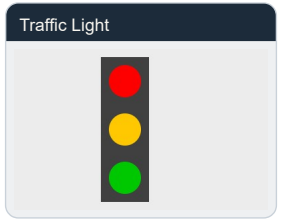
Scenario: a dark rectangle and three colored circles. Order matters: later shapes paint over earlier ones.

```
import javax.swing.*;
import java.awt.*;

class LightPanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.DARK_GRAY);
        g.fillRect(60, 20, 60, 160);
        g.setColor(Color.RED);
        g.fillOval(70, 30, 40, 40);
        g.setColor(Color.YELLOW);
        g.fillOval(70, 80, 40, 40);
        g.setColor(Color.GREEN);
        g.fillOval(70, 130, 40, 40);
    }
}

public class TrafficLight {
    public static void main(String[] args) {
        JFrame f = new JFrame("Traffic Light");
        f.add(new LightPanel());
        f.setSize(190, 240);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. **Painter's algorithm —** we draw back to front: the body first, the lights on top.
2. **Oval = a box —** fillOval(70, 30, 40, 40) fits a circle into a 40 x 40 square.
3. **Same object, many calls —** one Graphics g, one setColor before each shape.

Drawing Text: Score and Lives

Scenario: the game needs a score. We draw it, not a JLabel.

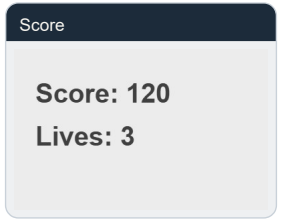
```
import javax.swing.*;
import java.awt.*;

class ScorePanel extends JPanel {
    private int score = 120, lives = 3;

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.DARK_GRAY);
        g.setFont(new Font("Arial", Font.BOLD, 22));
        g.drawString("Score: " + score, 20, 40);
        g.drawString("Lives: " + lives, 20, 75);
    }
}

public class ScoreText {
    public static void main(String[] args) {
        JFrame f = new JFrame("Score");
        f.add(new ScorePanel());
        f.setSize(260, 150);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. State and view — score and lives are plain fields; paintComponent only displays them.

2. Baseline — the y in drawString is the bottom of the letters, not the top.

3. Encapsulation — the fields are private; later weeks add methods to change them.

Combining Shapes: a House

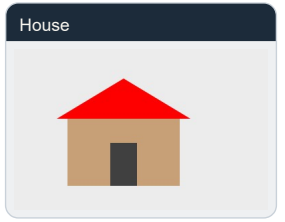
Scenario: a polygon roof, a rectangle wall and a door. Several shapes make one picture.

```
import javax.swing.*;
import java.awt.*;

class HousePanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(new Color(200, 160, 120));
        g.fillRect(60, 100, 120, 80);
        g.setColor(Color.RED);
        int[] xs = {50, 120, 190};
        int[] ys = {100, 50, 100};
        g.fillPolygon(xs, ys, 3);
        g.setColor(Color.DARK_GRAY);
        g.fillRect(105, 130, 30, 50);
    }
}

public class HouseDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("House");
        f.add(new HousePanel());
        f.setSize(250, 250);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. **Arrays as points —** fillPolygon takes an array of x values, an array of y values and a count.
2. **Draw order —** wall first, then the roof, then the door.
3. **Decomposition —** a complex picture is just a list of simple shapes.

Arcs: a Smiley Face

Scenario: the first curved line. drawArc takes a start angle and a sweep, in degrees.

```java
import javax.swing.*;
import java.awt.*;

class SmileyPanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.YELLOW);
        g.fillOval(50, 30, 100, 100);
        g.setColor(Color.BLACK);
        g.drawOval(50, 30, 100, 100);
        g.fillOval(80, 60, 10, 14);
        g.fillOval(110, 60, 10, 14);
        g.drawArc(75, 70, 50, 40, 200, 140);
    }
}

public class Smiley {
    public static void main(String[] args) {
        JFrame f = new JFrame("Smiley");
        f.add(new SmileyPanel());
        f.setSize(230, 190);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. Angles — 0 degrees is 3 o'clock; positive angles go counter-clockwise.

2. Start and sweep — 200 degrees plus a sweep of 140 draws the lower curve, a smile.

3. Two calls, one circle — fillOval colors it, drawOval adds the outline.

repaint(): Asking, Not Drawing

repaint() is a **request**. It returns at once; Swing calls paintComponent a moment later.



Swing also calls paintComponent when...	
the window is shown for the first time	you did nothing
the window is resized or uncovered	you did nothing
you call repaint()	this is the one you control

Never call paintComponent(g) yourself. You have no valid Graphics to give it.

Timer, Week 3 vs Week 4 (1/2): Same Timer, New Job

The Timer line is identical. What changes is what we ask it to do on each tick.

	Week 3: colored JLabel	Week 4: our own paintComponent
Creating the Timer	new Timer(16, e -> ...)	new Timer(16, e -> ...), the same
Last line of the listener	box.setLocation(x, y);	repaint();
Who draws the picture?	the JLabel, a ready-made component	our paintComponent(Graphics g)
What do we move?	a component, by position	nothing: we change numbers only
Where do x and y live?	static fields in main	private fields of the panel or of an Egg
Layout	needs null layout	no layout tricks needed
100 eggs	100 JLabels to manage	one panel that draws 100 shapes

Timer, Week 3 vs Week 4 (2/2): Where the Drawing Happens

Week 3: move a component

```
new Timer(16, e -> {
    x += dx;
    box.setLocation(x, 50);
}).start();
```

Timer → update x → setLocation → Swing repaints the label for us

Week 4: change numbers, request a repaint

```
new Timer(16, e -> {
    x += dx;
    repaint();
}).start();

// elsewhere in the panel:
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    g.fillOval(x, 50, 40, 40);
}
```

Timer → update x → repaint() → paintComponent(g) → we draw

The Timer never draws. It only updates numbers and calls repaint(). All drawing lives in paintComponent.

A Bouncing Ball with Timer + repaint()

Scenario: Week 3's bounce, redrawn. The logic is identical; only the last line and the drawing changed.

```
import javax.swing.*;
import java.awt.*;

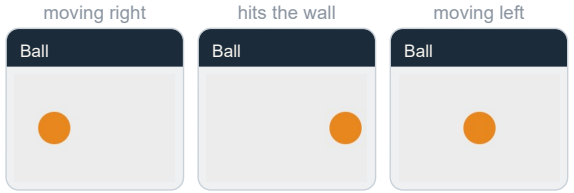
class BallPanel extends JPanel {
    private int x = 0, dx = 4;

    BallPanel() {
        new Timer(16, e -> {
            x += dx;
            if (x < 0 || x > getWidth() - 40) dx = -dx;
            repaint();
        }).start();
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillOval(x, 60, 40, 40);
    }
}

public class BallDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Bouncing Ball");
        f.add(new BallPanel());
        f.setSize(420, 200);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. State inside the object — x and dx are private fields of BallPanel, not static fields of main.

2. Timer in the constructor — the panel starts its own animation when it is created.

3. Separation — the listener only updates; paintComponent only draws.

A Falling Oval Egg

Scenario: Week 3's falling egg, now a real oval. It restarts at a random column.

```
import javax.swing.*;
import java.awt.*;

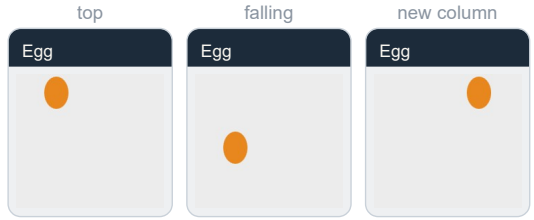
class FallPanel extends JPanel {
    private int x = 100, y = 0;
    private final java.util.Random rnd = new java.util.Random();

    FallPanel() {
        new Timer(16, e -> {
            y += 4;
            if (y > getHeight()) {
                y = 0;
                x = rnd.nextInt(getWidth() - 30);
            }
            repaint();
        }).start();
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.ORANGE);
        g.fillOval(x, y, 30, 40);
    }
}

public class FallDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Falling Egg");
        f.add(new FallPanel());
        f.setSize(260, 340);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. **One fewer component —** no JLabel, no setBounds, no null layout.
2. **Random once —** the Random is a field created once, not inside paintComponent.
3. **Reset rule —** identical to Week 3: y past the bottom means start again.

Keep State and Drawing Apart

The most useful habit of the week. Two jobs, two places.

update: the Timer listener

- changes x, y, score, lives
- checks edges and misses
- ends with repaint()
- never draws

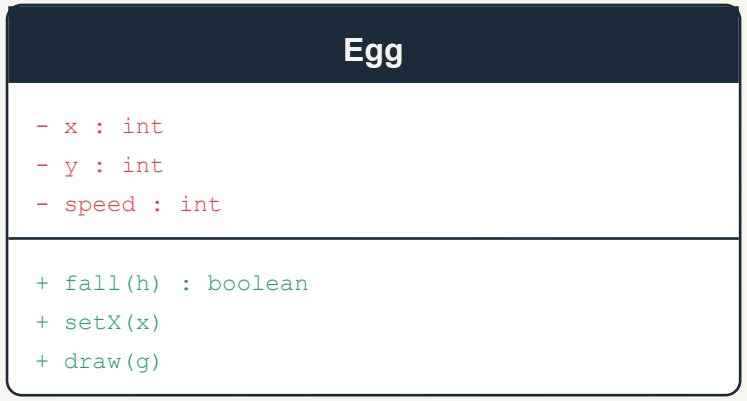
render: paintComponent

- reads x, y, score, lives
- draws them
- may run at any time (resize, uncover)
- never changes state

Because Swing may call paintComponent at any moment, changing the game inside it would make the game speed depend on window events.

OOP 4: The Egg Draws Itself

Week 3's Egg owned a JLabel. This one owns only numbers, and knows how to draw them.




```
NEEDS: java.awt.*
class Egg {
    private int x, y;
    private final int speed;

    Egg(int x, int speed) {
        this.x = x;
        this.speed = speed;
    }

    boolean fall(int fieldHeight) {
        y += speed;
        if (y > fieldHeight) { y = 0; return true; }
        return false;
    }

    void setX(int x) { this.x = x; }

    void draw(Graphics g) {
        g.setColor(Color.ORANGE);
        g.fillOval(x, y, 30, 40);
        g.setColor(Color.WHITE);
        g.fillOval(x + 7, y + 8, 8, 10);
    }
}
```

Encapsulation: x and y are private. **Responsibility:** the panel no longer knows what an egg looks like; the egg does.

Egg: Wrapping a JLabel vs. Drawing Directly with Graphics

Wrapping a component: Egg HAS a JLabel

```
private final JLabel view
    = new JLabel();

void fall() {
    y += speed;
    view.setLocation(x, y);
}
```

Depends on a Swing component. Needs null layout. The panel must add() every egg.

Drawing directly: Egg uses a Graphics

```
boolean fall(int h) {
    y += speed;
    ...
}

void draw(Graphics g) {
    g.fillOval(x, y, 30, 40);
}
```

Depends only on a Graphics to draw on. Easy to test, easy to keep in a list later.

Next weeks: many Egg objects in a list, each asked to fall() and draw(). The panel stays almost unchanged.

The Panel Delegates to the Egg

Scenario: the panel has one egg object. It asks the egg to move and to draw.

```
import javax.swing.*;
import java.awt.*;

class EggPanel extends JPanel {
    private final Egg egg = new Egg(120, 4);

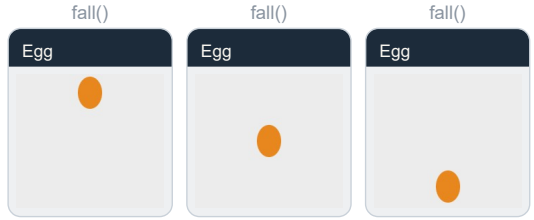
    EggPanel() {
        new Timer(16, e -> {
            egg.fall(getHeight());
            repaint();
        }).start();
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        egg.draw(g);
    }
}

public class EggPanelDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Egg Panel");
        f.add(new EggPanel());
        f.setSize(260, 340);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}

// Egg class from the previous slide goes in the same file
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. Delegation — the panel does not know how an egg looks; it just calls egg.draw(g).

2. Passing the Graphics — g is handed down: the panel gives its pen to the egg.

3. Add Egg.java to the file — copy the class from the earlier slide next to EggPanel.

Drawing the Basket (and an Egg in It)

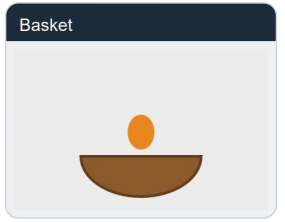
Scenario: the basket is the half of an oval. Its position comes from the panel height, so it follows resizing.

```
import javax.swing.*;
import java.awt.*;

class BasketPanel extends JPanel {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        int x = 90, y = getHeight() - 70;
        g.setColor(new Color(139, 90, 43));
        g.fillArc(x, y, 90, 60, 180, 180);
        g.setColor(new Color(100, 62, 28));
        g.drawArc(x, y, 90, 60, 180, 180);
        g.drawLine(x, y + 30, x + 90, y + 30);
        g.setColor(Color.ORANGE);
        g.fillOval(x + 30, y - 25, 30, 40);
    }
}

public class BasketDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Basket");
        f.add(new BasketPanel());
        f.setSize(280, 300);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



STEP BY STEP — WHERE OOP SHOWS UP

1. **fillArc —** start 180 plus sweep 180 is the lower half of the oval.
2. **Computed position —** y = getHeight() - 70 keeps the basket at the bottom.
3. **Next week —** x will come from the keyboard, so the player can move it.

A Smoother Pen: Graphics2D

```
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(
        RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON);
    g2.setStroke(new BasicStroke(4));
    g2.drawOval(40, 30, 100, 100);
}
```

Cast — Graphics2D extends Graphics, and the g Swing passes is really a Graphics2D. The cast unlocks the extra methods.

Anti-aliasing — smooths jagged edges on ovals and diagonal lines.

BasicStroke — line thickness for the draw... methods.

Giving the Panel a Size

An empty panel has no natural size. Tell it how big the game field should be.

```
class FieldPanel extends JPanel {
    FieldPanel() {
        setPreferredSize(new Dimension(400, 360));
        setBackground(Color.WHITE);
    }
    ...
}

// in main:
f.add(new FieldPanel());
f.pack();       // fit the window to the panel
```

call	meaning
setPreferredSize(d)	the size the panel would like
frame.pack()	size the window to fit it
frame.setSize(w, h)	fix the window size by hand

With pack(), the field is exactly 400 x 360, and the window frame and buttons are added around it.

The Pattern You Will Use All Semester

```
class GamePanel extends JPanel {
    // 1. STATE: private fields
    private int x = 0, dx = 4;

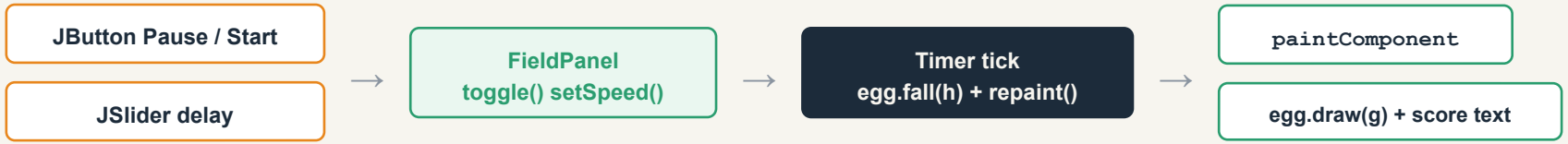
    GamePanel() {
        // 2. UPDATE: Timer changes state
        new Timer(16, e -> {
            x += dx;
            repaint();
        }).start();
    }

    // 3. RENDER: paintComponent draws state
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.fillOval(x, 60, 40, 40);
    }
}
```

1. **State** private fields hold the numbers.
2. **Update** the Timer listener changes them and calls repaint().
3. **Render** paintComponent draws what the numbers say.

Falling Egg v1: How the Pieces Connect

Same controls as v0, but the field is now our own FieldPanel and the egg draws itself.



The window code (main) only talks to two FieldPanel methods. It never touches the Timer directly.

Falling Egg v1, Part 1: The FieldPanel

Everything the field needs lives inside one class. The window only calls toggle() and setSpeed().

```
// Egg class from the earlier slide goes in this file too
class FieldPanel extends JPanel {
    private final Egg egg = new Egg(100, 4);
    private final java.util.Random rnd = new java.util.Random();
    private final Timer timer;
    private int missed = 0;

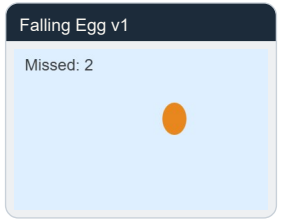
    FieldPanel() {
        setPreferredSize(new Dimension(400, 360));
        setBackground(new Color(222, 240, 255));
        timer = new Timer(16, e -> {
            if (egg.fall(getHeight())) {
                missed++;
                egg.setX(rnd.nextInt(getWidth() - 30));
            }
            repaint();
        });
        timer.start();
    }

    boolean toggle() {
        if (timer.isRunning()) timer.stop(); else timer.start();
        return timer.isRunning();
    }

    void setSpeed(int delay) { timer.setDelay(delay); }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        egg.draw(g);
        g.setColor(Color.DARK_GRAY);
        g.drawString("Missed: " + missed, 10, 20);
    }
}
```

OUTPUT



WHAT TO NOTICE

1. **Encapsulation —** timer is private; toggle() and setSpeed() are the only doors in.

2. **Inheritance + composition —** FieldPanel IS a JPanel and HAS an Egg and a Timer.

3. **State and view —** the listener updates, paintComponent draws.

Falling Egg v1, Part 2: The Window

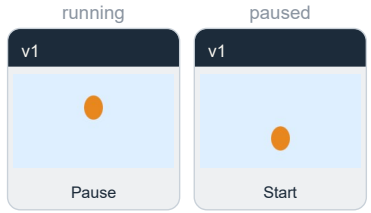
Put "import javax.swing.; import java.awt.*;" at the top of the file. The full file is in the lab notes.*

```
public class FallingEggV1 {
    public static void main(String[] args) {
        JFrame f = new JFrame("Falling Egg v1");
        FieldPanel field = new FieldPanel();
        JButton toggle = new JButton("Pause");
        JSlider speed = new JSlider(5, 40, 16);
        JPanel bar = new JPanel();
        bar.add(toggle);
        bar.add(new JLabel("Delay:"));
        bar.add(speed);

        toggle.addActionListener(e ->
            toggle.setText(field.toggle() ? "Pause" : "Start"));
        speed.addChangeListener(e -> field.setSpeed(speed.getValue()));

        f.add(field);
        f.add(bar, BorderLayout.SOUTH);
        f.pack();
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

OUTPUT



WHAT TO NOTICE

1. Interfaces again — one ActionListener for the button, one ChangeListener for the slider.

2. Clean boundary — main never sees the Timer, only the FieldPanel methods.

3. Compare with v0 — same behavior, far less tangled code.

Common Mistakes I: Nothing Appears

A typo in the method name or signature

Symptom: the panel stays blank. paintcomponent or paintComponent(Graphic g) is just a new, unused method.

Fix: always write @Override: the compiler then reports the mismatch.

The panel was never added to the frame

Symptom: an empty window.

Fix: frame.add(new GamePanel()); before setVisible(true).

The panel has no size

Symptom: with pack(), the window is a tiny sliver.

Fix: call setPreferredSize(new Dimension(w, h)) in the panel constructor.

Common Mistakes II: Drawing Misbehaves

Forgot repaint()

Symptom: x and y change, but the picture never moves.

Fix: end the Timer listener with repaint();

Forgot super.paintComponent(g)

Symptom: a trail of old frames, or garbage on resize.

Fix: make it the first line of paintComponent.

Called paintComponent yourself

Symptom: a compile error, or a flicker with a fake Graphics.

Fix: never call it. Call repaint() and let Swing do the rest.

Common Mistakes III: Coordinates and Cost

Treating x, y as the center

Symptom: shapes appear shifted right and down by half their size.

Fix: fillOval(x, y, w, h): x, y is the top-left corner of the box.

Creating objects inside paintComponent

Symptom: new Random() or new Color(...) 60 times a second.

Fix: create long-lived objects once, as fields.

Changing game state inside paintComponent

Symptom: the game speeds up when the window is resized.

Fix: update state only in the Timer listener.

Lab Goals I: Static Drawing (Easy)

- 1. First paint: your own JPanel subclass draws a rectangle
- 2. Traffic light: colors and ovals
- 3. House: polygon, rectangles and a door
- 4. Smiley: ovals and an arc
- 5. Scoreboard: drawString with a custom font

Lab Goals II: Animation and the Game (Medium to Hard)

- 6. Bouncing ball: Timer + repaint()
- 7. Falling oval egg with a random column
- 8. Egg class: the egg draws itself
- 9. Basket and egg together on one panel
- 10. Falling Egg v1: FieldPanel, button, slider, missed text
- 11. Challenge: three eggs with different speeds and colors, in an array

NEXT WEEK

Week 5: Keyboard & Mouse

The basket gets a player. KeyListener and key bindings, MouseListener and MouseMotionListener, the focus problem, and the first collision test: did the egg land in the basket?

Questions?